

List Snippets

< > localhost:3000

Home

Snippets New

List Files	View
Print Folders	View
Fetch Data	View

Create Snippet

< > localhost:3000/snippets/new

Home

Create Snippet

Title

Code

Save

View a Snippet

< > localhost:3000/snippets/1

Home

Print Files Edit Delete

```
const fs = require('fs');

const printFiles = () => {
  ....
}
```

Edit Snippet

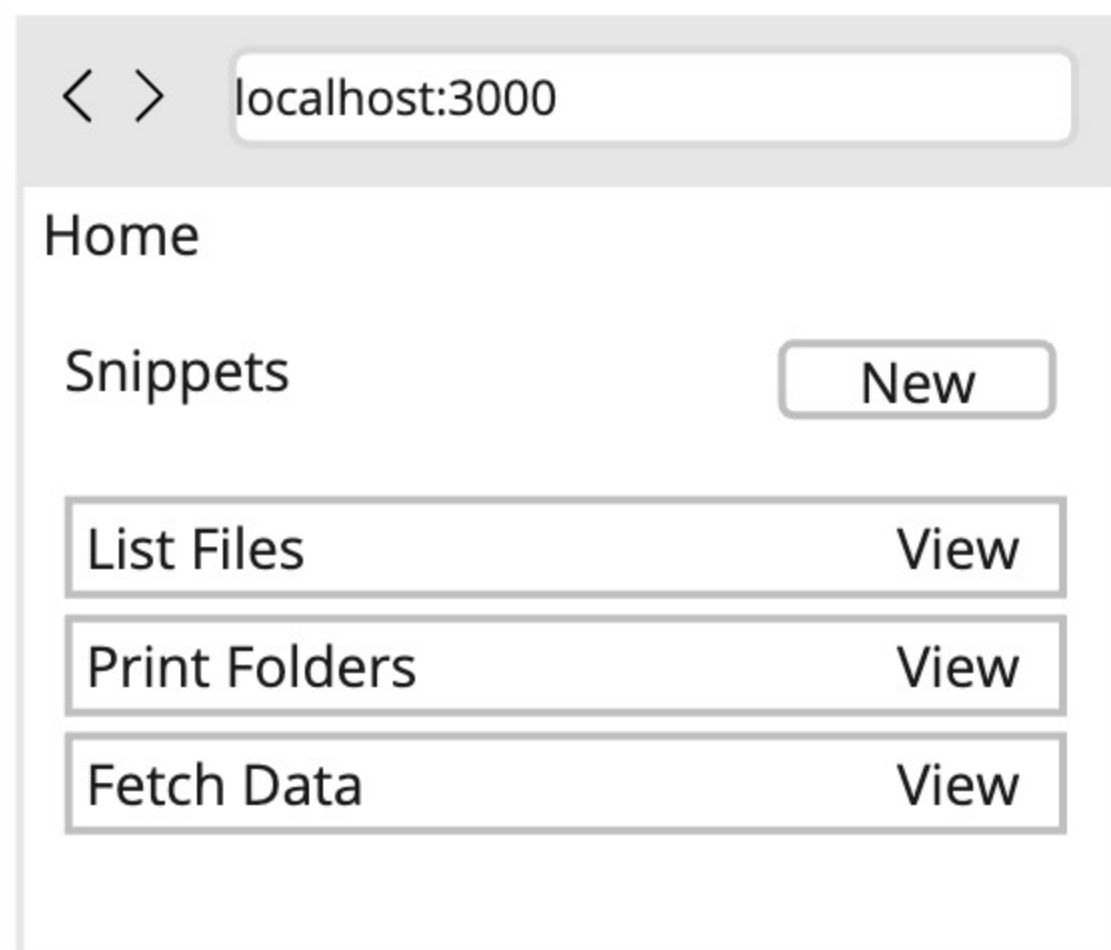
< > localhost:3000/snippets/1/edit

Home

Print Files

```
1 const fs = require('fs');
2 const path = require('path');
3
4 function printFilesAndFolders(dirPath, indent = '') {
5   const files = fs.readdirSync(dirPath);
6
7   for (const file of files) {
8     const filePath = path.join(dirPath, file);
```

Save



Next

prisma



**SQLite
Database**

Creating Records

1

Create a prisma client to access our database

2

Create a form in SnippetCreatePage

3

Define a *Server Action*. This is a function that will be called when the form is submitted

4

In the Server Action, validate the users input then create a new snippet

5

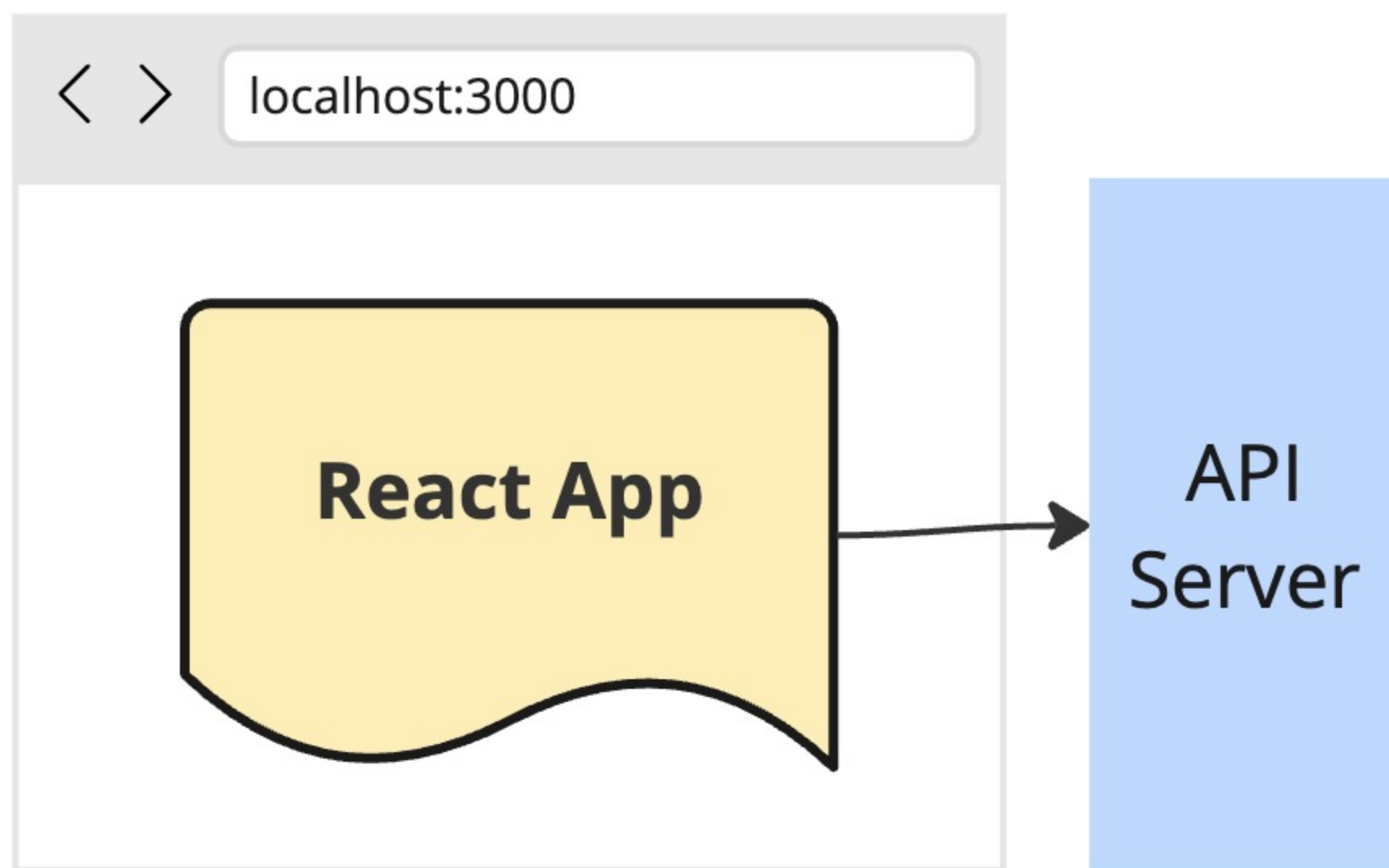
Redirect the user to the Home Page, which lists all the snippets

Server Actions

Number one way to **change data** in a next app

Super close integration with HTML forms

Server actions are functions that will be called with the values a user entered into a form



Traditional React App

All JS running in the browser

When you need to change data, you make an HTTP request to an outside server

Requests are usually done using functions like 'fetch' or the 'axios' library

Next Server

```
import { redirect } from 'next/navigation';
import { db } from '@/db';

export default function SnippetCreatePage() {
```

```
  return (
    <form action={createSnippet}>
```

```
    async function createSnippet(formData: FormData) {
      // This needs to be a server action!
      'use server';

      // Check the user's inputs and make sure they're valid
      const title = formData.get('title') as string;
      const code = formData.get('code') as string;

      // Create a new record in the database
      const snippet = await db.snippet.create({
        data: {
          title,
          code,
        },
      });
      console.log(snippet);

      // Redirect the user back to the root route
      redirect('/');
    }
  )
}
```

< >

Home

Create Snippet

Title

Code

Info From Form	
title	Add Numbers
code	function add() {

Next Server

Next receives form data

Next passes form data to our server action

We create a new snippet and redirect the user

Next sends a response back telling the app to show the home page

Fetching Data

1

Create a *server component* - a component that doesn't have 'use client' at the top

2

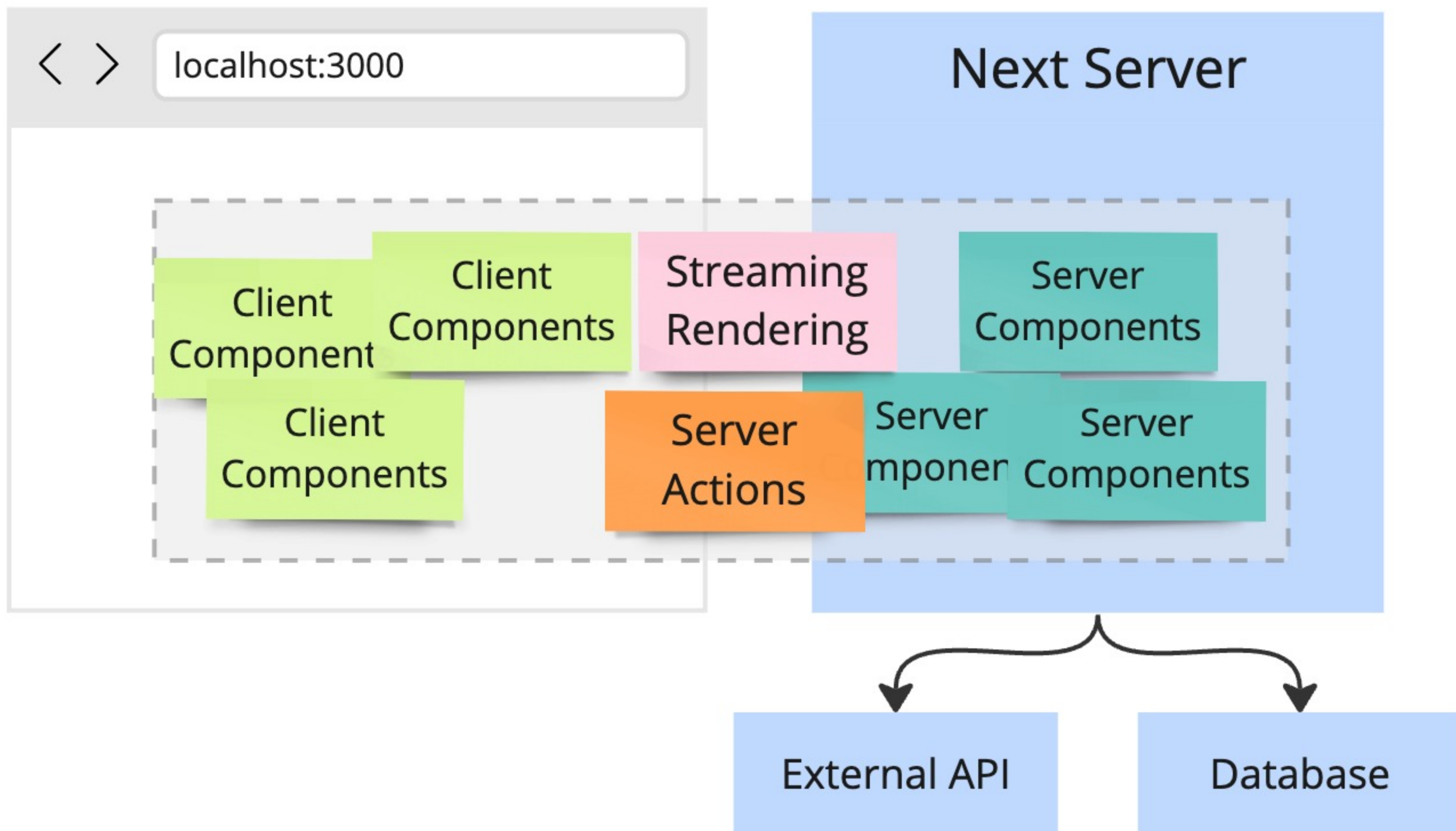
Mark the component as 'async'

3

Make an HTTP request or directly access a database to get your data

4

Render your data directly, or pass it to a child component



With Next, some of our code is running in the browser and some is running on the server

It can be challenging - yet important - to know where your code is running!

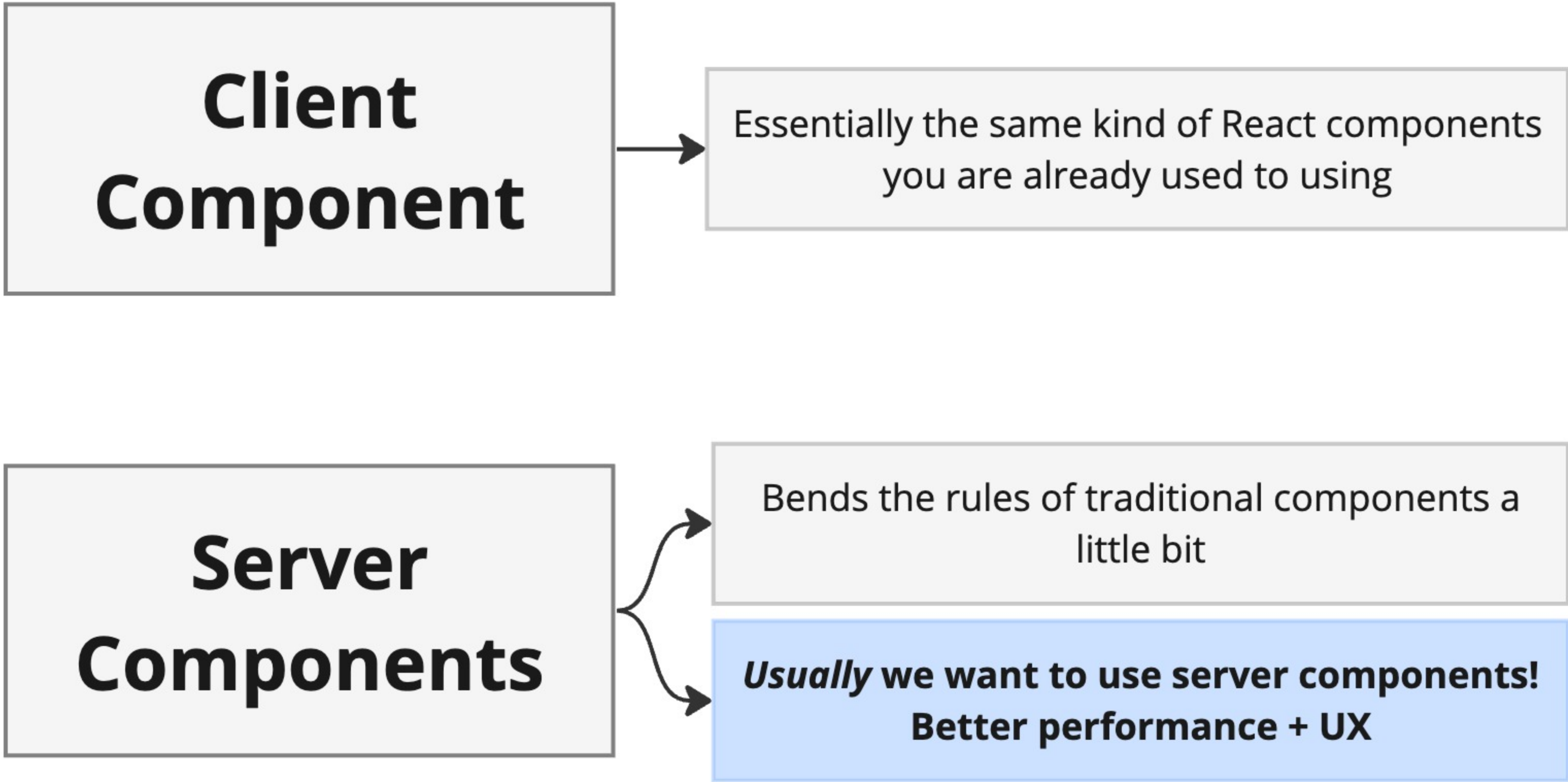
**Next apps are built with two kinds
of components**

```
graph TD; A[Next apps are built with two kinds of components] --> B[Server Components]; A --> C[Client Component];
```

**Server
Components**

**Client
Component**

Client Component



```
graph LR; A[Client Component] --> B[Essentially the same kind of React components you are already used to using]; C[Server Components] --> D[Bends the rules of traditional components a little bit]; C --> E["Usually we want to use server components! Better performance + UX"]
```

Essentially the same kind of React components you are already used to using

Server Components

Bends the rules of traditional components a little bit

Usually we want to use server components!
Better performance + UX

Server Components

```
1  export default async function HomePage() {  
2    const res = await fetch('...');  
3    const data = await res.json();  
4  
5    return (  
6      <div>  
7        {data}  
8      </div>  
9    );  
10 }
```

By default, all components are server components

We want to use server components. Tons of UX, performance, etc benefits by using them

Can use async/await! Don't need useState or useEffect to do data fetching!

Have a few limitations

Server Components

Can **not** use any kind of hook

```
1 import { useState } from "react";
2
3 export default async function HomePage() {
4   const [color, setColor] = useState("");
5
6   return <div>{color}</div>;
7 }
```

Failed to compile

```
./src/app/page.tsx
ReactServerComponentsError:

You're importing a component that needs useState. It only works in
a Client Component but none of its parents are marked with "use
client", so they're Server Components by default.
Learn more: https://nextjs.org/docs/getting-started/react-
essentials

    at [Users/stephengrider/workspace/next-workspace/prod/snippets-
read/src/app/page.tsx:1:1]
    1 | import { useState } from 'react';
      |
```

Can **not** assign any event handlers

```
1 export default async function HomePage() {
2   return (
3     <button
4       onClick={() => console.log('hi')}
5     />
6   );
7 }
```

1 of 1 unhandled error Next.js (14.0.0) out of date (learn more) X

Unhandled Runtime Error

Error: Event handlers cannot be passed to Client Component props.

```
<button onClick={function}>
  ^^^^^^^^^
```

If you need interactivity, consider converting part of this to a Client Component.

Client Components

```
1  'use client';
2
3  import { useState } from 'react';
4  import ServerComponent from 'alskdfj'
5
6  export default function Color() {
7    const [color, setColor] = useState('');
8
9    return (
10      <div>
11        <ServerComponent />
12        <input
13          value={color}
14          onChange={e => setColor(e
15                                .target.value)}
16        {color}
17      </div>
18    );
19  }
```

Created by adding 'use client' at the very top of the file

Have all the usual rules of components

Can use hooks, event handlers, etc

Can **not** directly show a Server Component (there is one exception)

OK, so when we do use each kind of component?

Use a client component if...

1

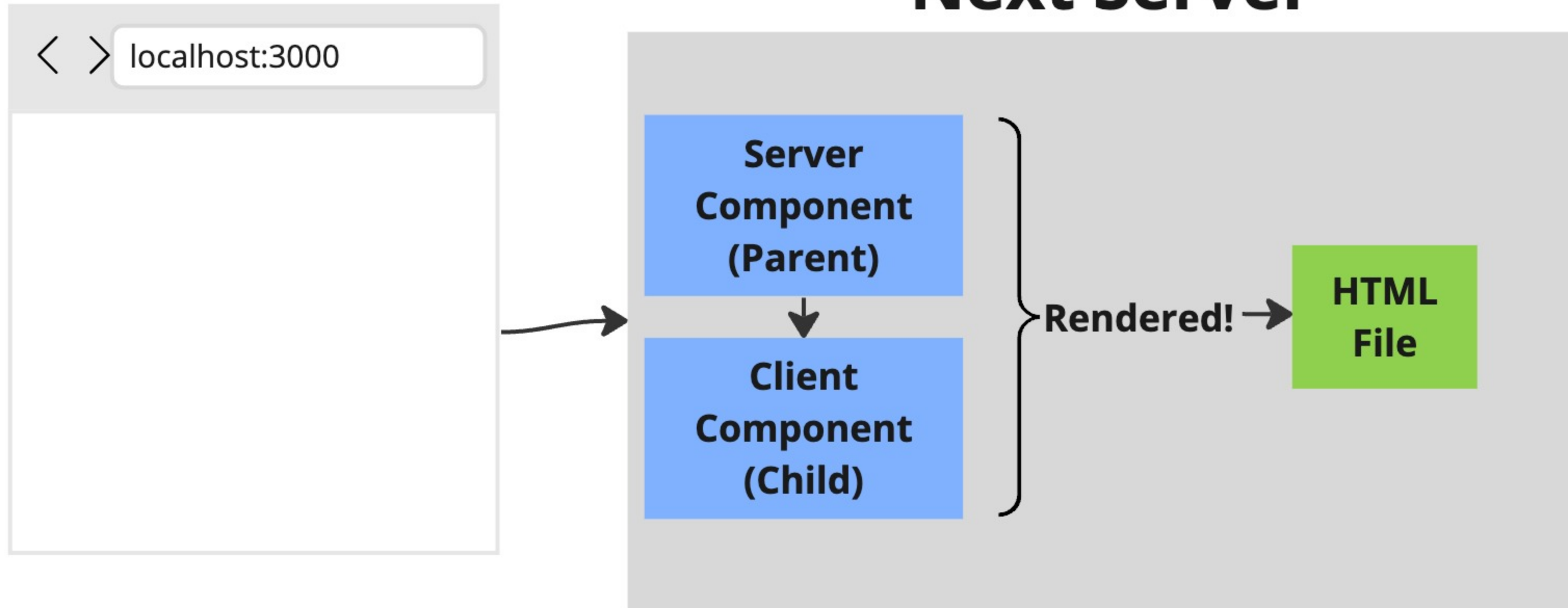
You need to use hooks

2

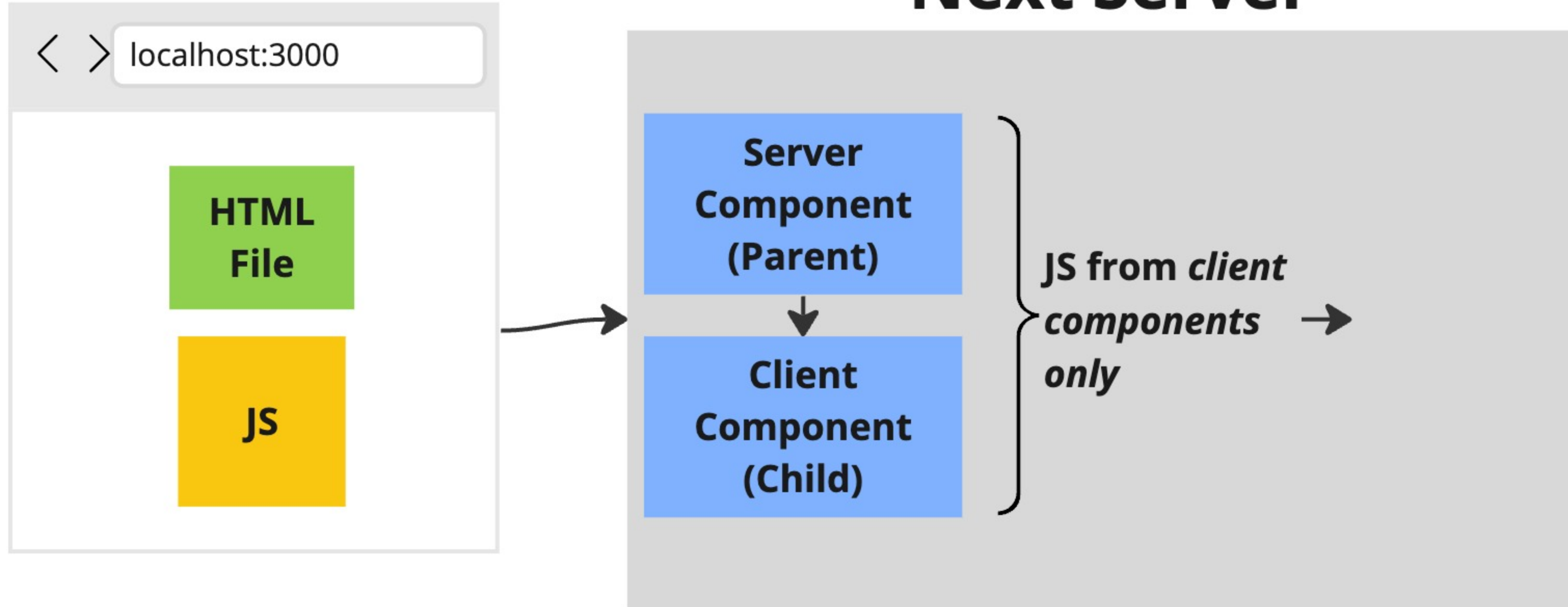
You need to use event handlers

There are some other situations where you will want to use a client component - we will see these later

Next Server

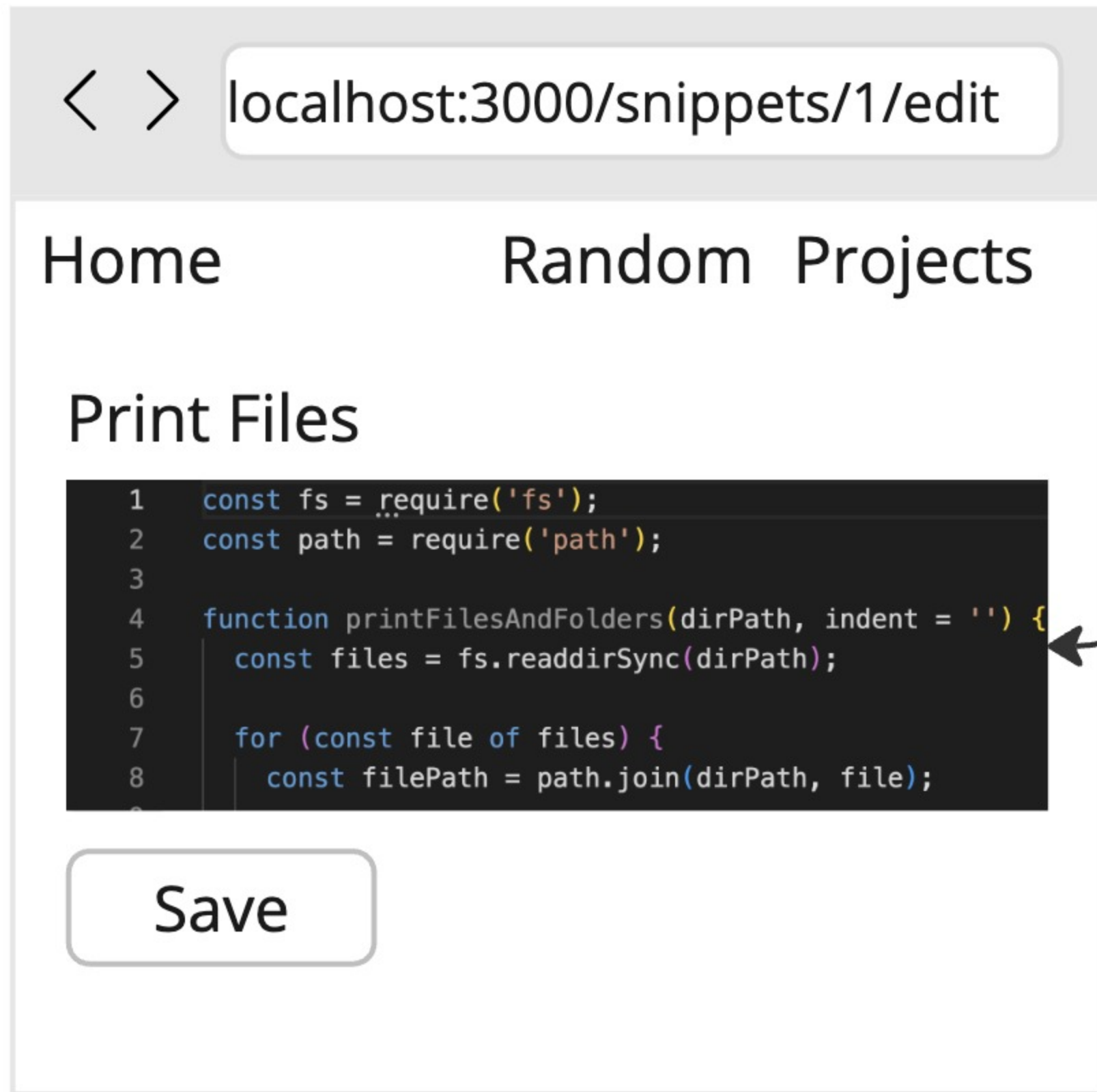


Next Server



Special File Names in the 'app' Folder

Name	Purpose
page.tsx	Displays the primary content for the page
layout.tsx	Wraps up the currently displayed page. Use to show content common across many pages
not-found.tsx	Displayed when we call the 'notFound' function
loading.tsx	Displayed when a server component is fetching some data
error.tsx	Displayed when an error occurs in a server component
route.tsx	Defines API endpoints



React Monaco Editor

Requires hooks + event handlers to work correctly



We need a **Client Component**

But we still need to fetch data...still need a **Server Component** somewhere!

SnippetEditPage
Server Component

snippet

SnippetEditForm
Client Component

Monaco Editor

npmjs.com/package/@monaco-editor/react

*Documentation for the Monaco
Editor React component*

```
'use client';

export default function SnippetEditForm({ snippet }) {

  async function updateSnippet() {
    'use server';

    // Code to update the snippet
  }

  return (
    <div>
      <Editor />
    </div>
  );
}
```

Server actions **cannot**
be defined in
Client Components

Server Actions in Client
Components
Option #1



Define the **Server Action** in
a **Server Component** and
pass it as **props** to the
Client Component

*Server components can't
pass event handlers down to
client components - this is
the one exception*

SnippetEditPage *Server Component*

```
export default function SnippetEditPage() {  
  async function updateSnippet(id, code) {  
    'use server';  
  
    // Code to update the snippet  
  }  
  
  return (  
    <SnippetEditForm onSubmit={updateSnippet} />  
  );  
}
```

onSubmit

snippet

SnippetEditForm *Client Component*

Monaco Editor

Server Actions in Client
Components
Option #2



Define the **Server Action** in
a **separate file** and **import**
it into the Client Component

actions.ts

```
'use server';  
  
export async function updateSnippet(id, code) {  
  // code to update the snippet  
  
  // code to redirect the user  
}
```

SnippetEditForm
Client Component

*Making server actions in a separate file?
You only have to write 'use server' once at
the top of the file*

Calling Server Actions from **Server** Components

Option #1

```
function ServerComponent() {
  async function editSnippet(formData) {
    'use server';

    const code = formData.get('code');
    // do something on the server
  }

  return <form action={editSnippet}>
    <input name="code" />
    <button type="submit">Submit</button>
  </form>
}
```

Option #2

```
function ServerComponent() {
  async function editSnippet(formData) {
    'use server';

    const code = formData.get('code');
    // do something on the server
  }

  async function deleteSnippet() {
    'use server';
  }

  return <form action={editSnippet}>
    <input name="code" />

    <button type="submit">Submit</button>
    <button
      type="submit"
      formAction={deleteSnippet}
    >
      Delete
    </button>
  </form>
}
```

Calling Server Actions from **Client** Components

Option #1

Client Component

```
'use client'

import * as actions from 'actions';

function ClientComponent() {
  const [code, setCode] = useState('');

  // Make sure 'editSnippet' gets called
  // with the code from editor
  const editSnippetAction =
    actions.editSnippet
      .bind(null, code);

  return <form action={editSnippetAction}>
    <button type="submit">Submit</button>
  </form>
}
```



```
'use server'

export async function editSnippet(code: string, formData: FormData) {
  // update snippet
}
```

We used 'bind' to customize the arguments

First argument will be the bound argument

Second argument will be 'FormData', containing data form (if any exists)

Calling Server Actions from **Client** Components

Option #2

Client Component

```
'use client'

import { startTransition } from 'react';
import * as actions from 'actions';

function ClientComponent() {
  const [code, setCode] = useState('');

  const handleClick = () => {
    startTransition(async () => {
      await actions.editSnippet(code);
    });
  };

  return <button onClick={handleClick}>
    Submit
  </button>
}
```

```
'use server'

export async function editSnippet(code: string) {
  // update snippet
}
```

Not tied to a 'form', so only the arguments passed *directly* to the server action are received

Makes sure we have updated data before we attempt to navigate

Client Component

```
'use client'

import * as actions from 'actions';

function ClientComponent() {
  const [code, setCode] = useState('');

  // Make sure 'editSnippet' gets called
  // with the code from editor
  const editSnippetAction = editSnippet
    .bind(null, code);

  return <form action={editSnippetAction}>
    <button type="submit">Submit</button>
  </form>
}
```



```
'use server'

export async function editSnippet(
  code: string,
  formData: FormData
) {
  // update snippet
}
```

Option #1

Form will work even if the user isn't running Javascript in their browser. *(pointless in this case since our form requires JS for the monaco editor)*

Client Component

```
'use client'

import * as actions from 'actions';

function ClientComponent() {
  const [code, setCode] = useState('');

  const handleClick = async () => {
    await actions.editSnippet(code);
  };

  return <button onClick={handleClick}>
    Submit
  </button>
}
```



```
'use server'

export async function editSnippet(
  code: string
) {
  // update snippet
}
```

Option #2

No messing around with 'bind' function

Closer to classic React behavior

Caching

Next implements caching in several locations.

Can lead to unexpected behavior

Data Cache

Responses from requests made with '**fetch**' are stored and used across requests.

Router Cache

'Soft' navigation between routes are cached in the browser and reused when a user revisits a page.

Request Memoization

Make two or more 'GET' requests with 'fetch' during a user's request to your server? Only one 'GET' is actually executed.

Full Route Cache

At build time, Next decides if your route is **static** or **dynamic**. If it is static, the page is rendered and the result is stored. In production, users are given this pre-rendered result.

Help! My next page is rendering with out-of-date data!

There are several ways to control caching

Time-Based



Every X seconds, ignore the cached response and fetch new data

On-Demand



Forcibly purge a cached response

**Disable
Caching**



Don't do any caching at all

On-Demand



Forcibly purge a cached response

Option #1

Dump cache for everything in a page

```
import { revalidatePath } from "next/cache";

// When we think data that the '/random'
// route uses has changed...
revalidatePath('/random');
```

Option #2

Dump cache for all data with a certain tag

```
import { revalidateTag } from "next/cache";

// When we think data with tag 'snippets'
// has changed...
revalidateTag('snippets');

export default async function Page() {
  await fetch('/snippets/random', {
    next: { tags: ['snippets'] }
  })
}
```

< > localhost:3000/snippets/new

Home

Create Snippet

Title

Code

Title must be at least 3 characters

Save

Error Handling with Server Actions

Remember, a big point of forms is that they can work without JS in the browser

Right now, forms in our pages are sending info **to** a server action

We need to somehow communicate info **from a server action back to a page**

React-dom (not react) contains a hook called 'useFormState' specifically for this

How a Server Form works without useFormState

**Page
Component**

< > localhost:3000/snippets/new

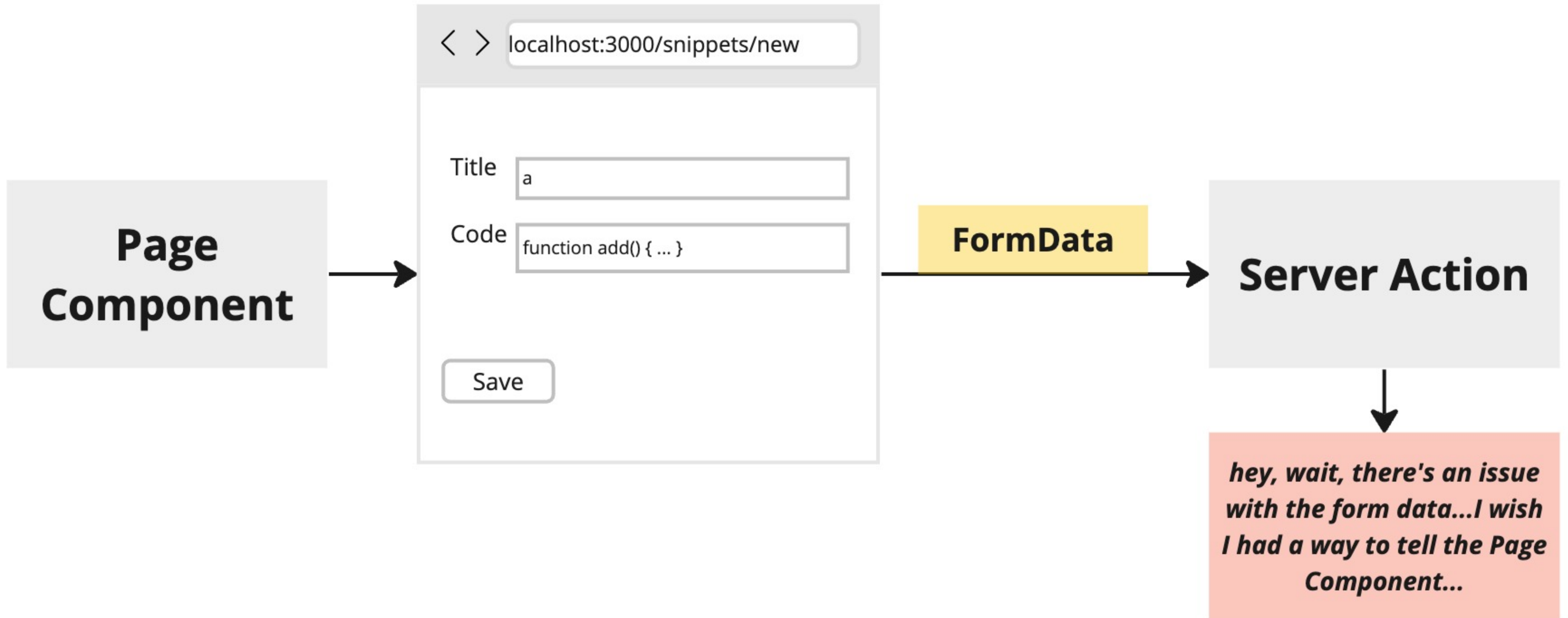
Title

Code

FormData

Server Action

How a Server Form works without useFormState



With useFormState

